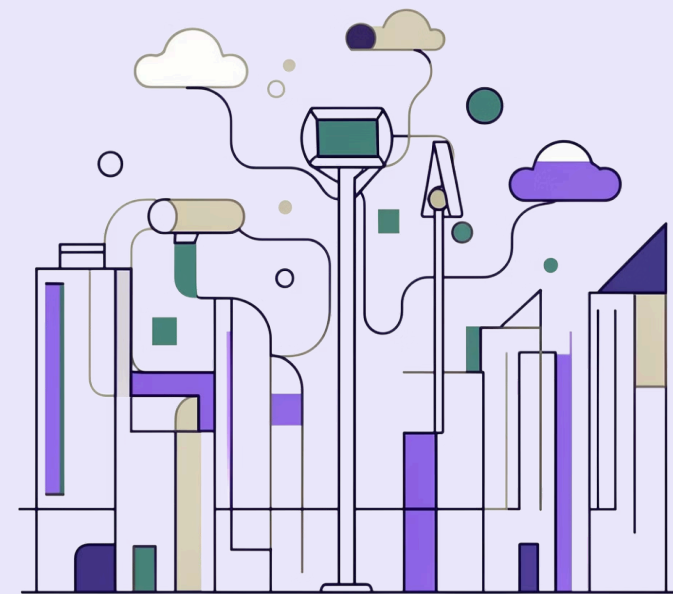


+ ClimApp

Aplicación web para consultar clima en tiempo real usando GPS y datos oficiales de AEMET. Desarrollada con Python y Flask, integra geolocalización del usuario, arquitectura por capas, persistencia SQL y una API REST documentada con Swagger.

PROYECTO 3 · BOOTCAMP IA

FLASK · FASTAPI · AEMET



El problema que resuelve

El reto

Muchas aplicaciones muestran datos meteorológicos genéricos por ciudad, pero no explican de dónde provienen los datos ni permiten ver el flujo técnico completo. El usuario no sabe qué estación se está usando ni a qué distancia se encuentra.

La solución de +ClimApp

La aplicación conecta piezas reales de desarrollo para ofrecer datos climáticos localizados y trazables:

- Ubicación GPS real del usuario
- Datos oficiales de AEMET OpenData
- Estación meteorológica más cercana
- Normalización de datos para el frontend
- Persistencia en base de datos SQL
- API REST documentada para pruebas

 La aplicación no muestra una temperatura fija: toma coordenadas reales, consulta AEMET, procesa los datos y devuelve una respuesta preparada para la interfaz y para otros endpoints.

Objetivos del proyecto

Objetivos técnicos

- Aplicación web funcional con Flask
- Conexión con la API externa de AEMET
- GPS para obtener ubicación del usuario
- Código organizado por capas
- Persistencia climática en SQL
- Endpoints REST con FastAPI
- Documentación y pruebas en Swagger
- Tests automáticos de la lógica principal

Objetivos de aprendizaje

- Trabajar con APIs externas reales
- Manejar errores y respuestas incompletas
- Separar responsabilidades dentro del código
- Usar SQLAlchemy para persistencia
- Validar datos con schemas Pydantic
- Entender el flujo completo entre frontend, backend y servicios

Arquitectura general por capas

La aplicación está organizada por responsabilidades para evitar que todo esté mezclado en un solo archivo. Cada capa tiene una función concreta, lo que hace el proyecto más mantenible y más fácil de probar.



Presentación

templates/ · static/ — Páginas HTML, CSS, JavaScript del frontend y captura de geolocalización.



Controladores

controllers/ — Rutas Flask, validación de parámetros y coordinación entre vista, servicios y persistencia.



Servicios

services/ — Lógica pesada: conexión con AEMET, normalización, alertas y reintentos.



Persistencia

db/ · repositories/ · schemas/ — Modelos SQLAlchemy, CRUD, validación Pydantic y repositorios JSON/SQLite.



API REST

api/ — Endpoints FastAPI para zonas y mediciones, documentados automáticamente en Swagger.

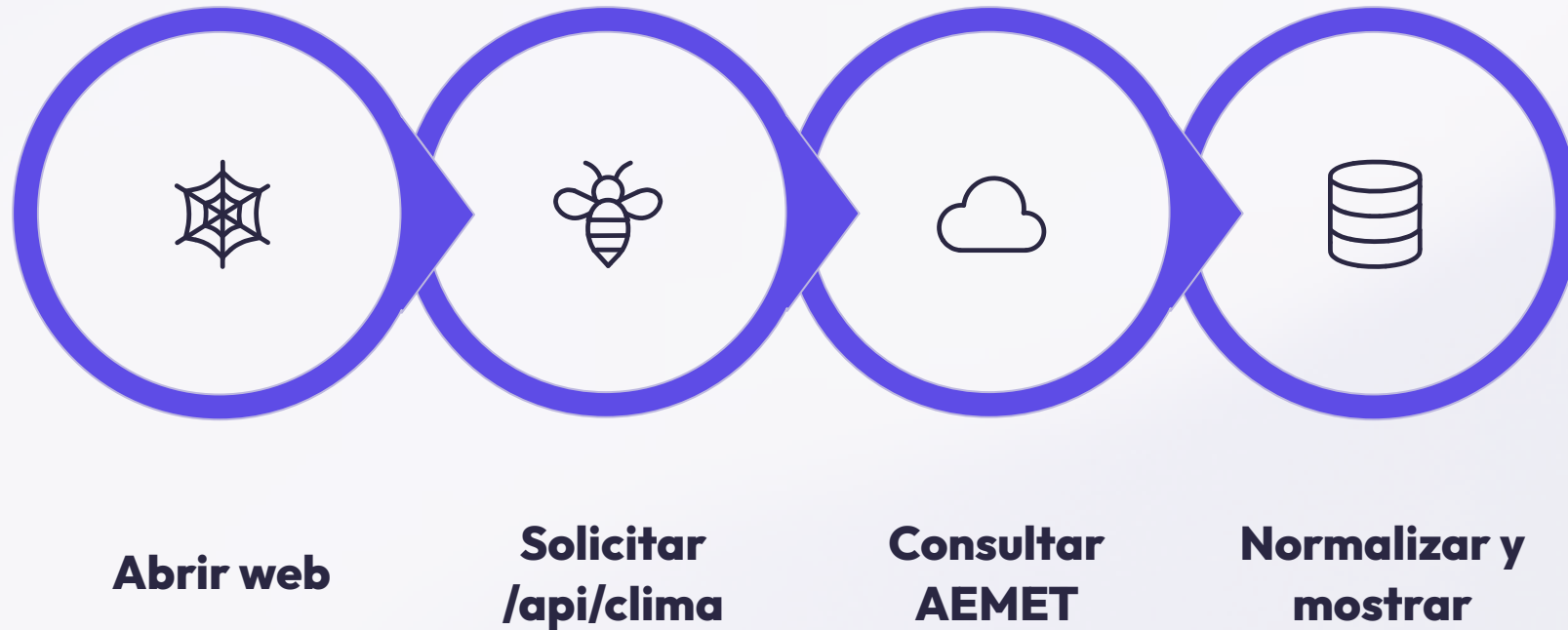


Tests

tests/ — Pruebas automáticas de controladores, servicios, repositorios y reglas de negocio con pytest.

Flujo principal: GPS + AEMET

El usuario solo ve una pantalla sencilla, pero por debajo hay varias capas coordinadas para obtener datos reales, transformarlos y mostrarlos. Este flujo es el centro del proyecto.



- ✓ Resultado final: el usuario recibe un JSON limpio con ciudad, temperatura, humedad, viento, lluvia, municipio, código de municipio, distancia a la estación y alertas.

Frontend e interfaz web

Pantalla principal

La interfaz muestra los datos climáticos de forma clara y directa tras aceptar el permiso de ubicación GPS:

- Municipio o estación detectada
- Temperatura, humedad, viento y lluvia
- Fuente de datos y estado visual del clima
- Alertas generadas (VERDE, ROJA_CALOR, ROJA_VIENTO...)

Página de prueba técnica

Se creó una página auxiliar en `http://127.0.0.1:5000/api` que permite:

- Probar la conexión GPS con AEMET
- Introducir coordenadas manuales
- Ver la respuesta JSON en bruto
- Abrir Swagger de FastAPI directamente

❏ El frontend está pensado para que la experiencia sea simple: aceptar ubicación y ver datos climáticos. La página de API sirve como herramienta de demostración técnica.

Backend Flask y conexión con AEMET

App principal Flask

app.py levanta la aplicación y registra los blueprints: vistas HTML, login/registro, API de clima y scheduler. El endpoint principal es:

```
GET /api/clima?lat=40.4168&lon=-3.7038
```

Este endpoint recibe coordenadas, las valida, llama al servicio meteorológico, normaliza la respuesta, guarda un resumen en SQL y devuelve JSON al frontend.

Cliente AEMET

services/aemet_client.py encapsula todas las llamadas a AEMET. Mantiene solo las conexiones necesarias:

- Observaciones actuales de todas las estaciones
- Catálogo de municipios
- Predicción diaria por municipio
- Predicción horaria por municipio

El cliente controla errores concretos: timeout, error HTTP, error de conexión, respuestas JSON inválidas y payloads con formato inesperado. Se eliminó el `except Exception` genérico para dejar logs más claros.

 AEMET usa un modelo de **doble petición**: la primera devuelve una URL temporal y la segunda descarga el JSON real. Este flujo vive en `_get_payload_from_endpoint()` dentro de `aemet_client.py`.

Servicios y normalización de datos

WeatherAPIService

`services/weather_api_service.py` orquesta el flujo meteorológico completo: convierte coordenadas a float, pide observaciones a AEMET, calcula la estación más cercana, obtiene el municipio asociado mediante `MunicipalityService`, pide predicciones diaria y horaria, y devuelve un diccionario enriquecido.

MunicipalityService

`services/municipality_service.py` resuelve el problema de que AEMET no acepta coordenadas directas para predicciones: obtiene el catálogo de municipios, lo cachea durante la ejecución, calcula el municipio más cercano por distancia geográfica y normaliza códigos como `id28079`.

NormalizerService

`services/normalizer_service.py` transforma los campos técnicos de AEMET (`ta`→temperatura, `hr`→humedad, `vv`→viento, `prec`→lluvia, `fint`→fecha) en un formato estable para el frontend. Incorpora coordenadas, distancia a la estación, municipio detectado, predicciones y alertas.

AlertService y RetryService

`alert_service.py` evalúa umbrales meteorológicos y genera alertas: temperatura muy alta, helada, viento fuerte, lluvia intensa o estado VERDE. `retry_service.py` proporciona una sesión HTTP con reintentos para mejorar la estabilidad frente a latencias o límites de la API de AEMET.

Persistencia y base de datos

Capa db/ con SQLAlchemy

La capa db/ contiene tres archivos con responsabilidades separadas:

- **database.py**: conexión, engine y sesiones SQLAlchemy
- **models.py**: modelos SQL — Zona y Medicion con claves primarias y foráneas
- **crud.py**: funciones create, read, update y delete

La relación entre modelos es **1 Zona → N Mediciones**. Una zona puede tener muchas mediciones; cada medición pertenece a una sola zona.

Datos guardados al consultar /api/clima

Cuando se ejecuta una consulta climática, la aplicación intenta guardar automáticamente un resumen con:

- Municipio y código INE
- Estación AEMET utilizada
- Fecha de la consulta
- Temperatura, humedad, viento y lluvia

- ✓ La base de datos permite conservar información estructurada para futuras consultas o histórico, aunque AEMET sea la fuente original.

API REST con FastAPI y Tests automáticos

FastAPI + Swagger

FastAPI se arranca con:

```
python -m uvicorn api.main:app --reload
```

Y se prueba en `http://127.0.0.1:8000/docs`. Los endpoints principales son:

- **/health/** — comprueba que la API funciona
- **/zonas/** — CRUD de zonas climáticas
- **/mediciones/** — CRUD de mediciones

Swagger permite probar los endpoints desde el navegador sin necesidad de Postman ni herramientas externas.

Suite de tests: 97 passed

El proyecto incluye tests automáticos para todas las capas:

- Cliente AEMET y servicio de clima
- Servicio de municipios y normalizador
- Repositorios y validadores
- Endpoints de zonas y mediciones
- Controladores

Resultado de ejecutar `pytest -q`:

✅ **97 passed** — Los tests fueron clave para detectar errores tras los merges y corregir conflictos de integración.

Cambios importantes y dificultades encontradas

Mejoras realizadas

- Mejora del manejo de errores en AemetClient (errores concretos en lugar de `except Exception`)
- Eliminación de código AEMET no necesario
- Creación de página `/api` para probar GPS y endpoints
- Ajuste de rutas de zonas para usar funciones CRUD correctas
- Compatibilidad de imports con `mas_climapp.main`
- Corrección de tests de API después del merge
- Limpieza de cachés y `.gitignore`

Retos técnicos

- Gestión de conflictos de merge en GitHub
- Diferencia entre Flask y FastAPI dentro del mismo proyecto
- Manejo de datos reales de AEMET con respuestas incompletas o inesperadas
- Persistencia entre tests por el uso de SQLite local
- Diferenciar datos generados localmente de cambios reales de código

⚠ Una parte importante del aprendizaje fue entender que un proyecto real no solo falla por el código de negocio, sino también por integraciones, merges, entornos, dependencias y datos generados.

Demo sugerida para clase

01

Abrir la app Flask

Navegar a `http://127.0.0.1:5000/`, aceptar el permiso GPS y mostrar los datos climáticos cargados desde AEMET en la pantalla principal.

03

FastAPI y Swagger

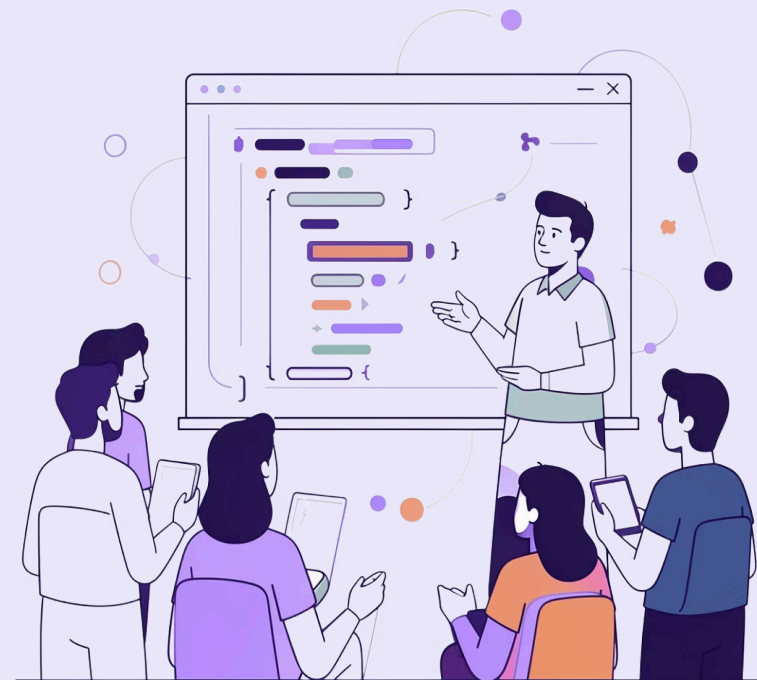
Arrancar FastAPI con `uvicorn` y abrir `http://127.0.0.1:8000/docs`. Probar `/health/`, `/zonas/` o `/mediciones/` directamente desde el navegador.

- ❏ La demo debe mostrar primero la experiencia de usuario y después la parte técnica. Así se entiende tanto el producto como la arquitectura.

02

Página de prueba técnica

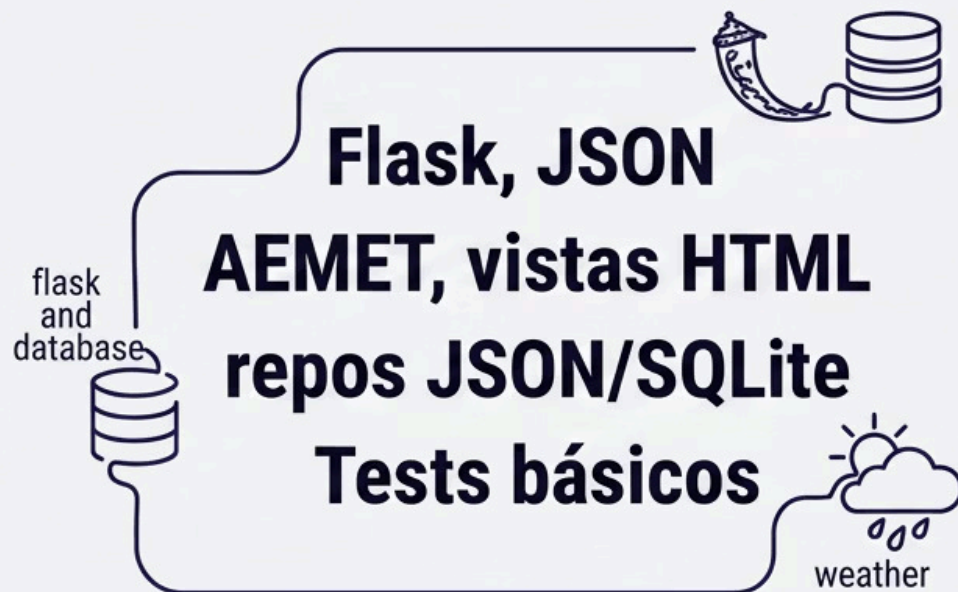
Abrir `http://127.0.0.1:5000/api`, probar coordenadas manuales o GPS y mostrar la respuesta JSON en bruto.



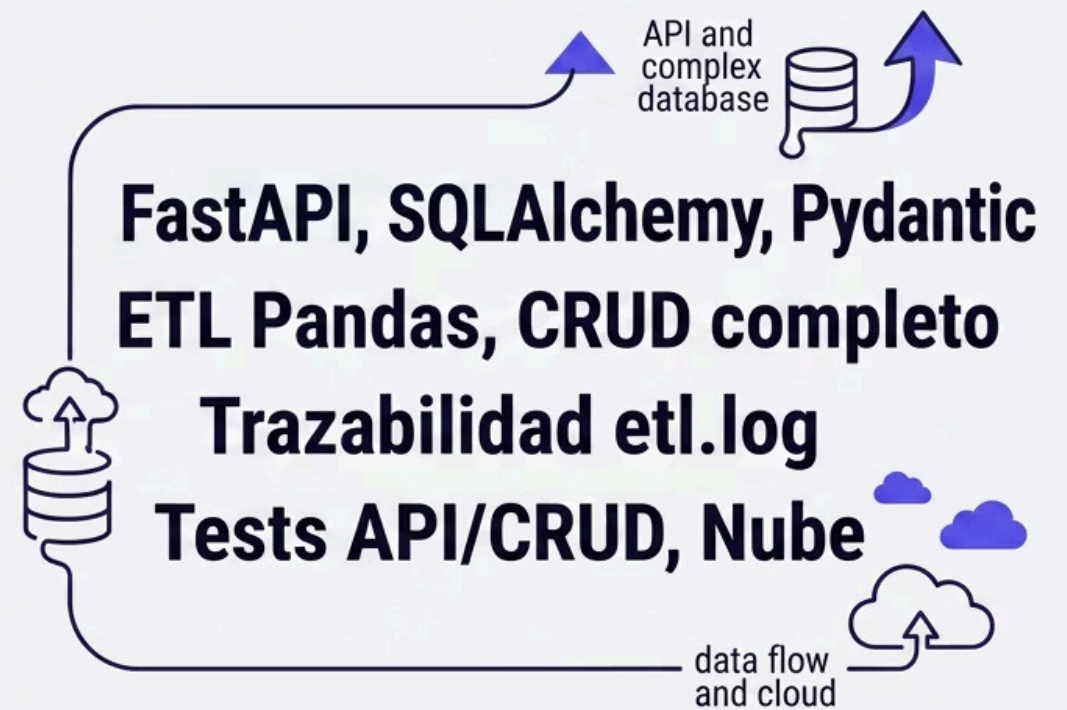
Evolución hacia el Proyecto 3

El Proyecto 3 no empieza desde cero: evoluciona ClimApp del Proyecto 2 añadiendo una capa profesional con ETL, base de datos relacional, Pydantic, FastAPI, CRUD, trazabilidad y tests.

PROYECTO 2



PROYECTO 3



Ya existe (P2)

app.py, controllers/, services/, repositories/, models/, templates/, static/, tests/ básicos, data/

Adaptar (P3)

api_controller.py, normalizer_service.py, sqlite_repository.py, requirements.txt, validators.py

Nuevo (P3)

main_api.py, db/, schemas/, etl/, api/routes_*, logs/etl.log, services/etl_log_service.py, tests/test_etl.py, test_crud.py, test_api_mediciones.py

Reparto de tareas del equipo

Reparto ajustado para un equipo de 6 personas. Cada persona tiene una zona clara, pero el trabajo encaja como piezas de LEGO: base de datos → ETL → validación → API → tests → documentación.

Persona	Rol	Responsabilidades principales
Gianmario	Desarrollador BD	db/database.py, db/models.py — Conexión a la BD, tablas Zona y Medicion, claves primarias y foráneas
Luis	Desarrollador CRUD	db/crud.py, repositories/sqlite_repository.py — Crear, leer, actualizar y borrar mediciones; adaptar persistencia heredada
Anas	Desarrollador ETL	etl/extract.py, transform.py, load.py, etl_runner.py — Limpieza de nulos, tipos y duplicados con Pandas; carga a SQL
Elena	Scrum Master / API	main_api.py, api/routes_mediciones.py, api/routes_zonas.py — GET, POST, PUT/PATCH, DELETE con códigos 201, 404 y errores claros
Gema	Desarrolladora Pydantic	schemas/zona_schema.py, schemas/medicion_schema.py, utils/validators.py, services/alert_service.py — Validar entradas y evitar datos malformados
Verónica	Product Manager / QA	tests/test_etl.py, test_crud.py, test_api_mediciones.py, logs/etl.log, README — Tests, linaje del dato, diagrama ER y presentación

Conclusión y mejoras futuras

Lo que demuestra Mas ClimApp

La aplicación integra varias piezas reales de desarrollo en un flujo completo y coherente:

- **Frontend con GPS**

Captura de coordenadas reales del usuario desde el navegador.

- **Backend Flask + FastAPI**

Arquitectura por capas con controladores, servicios y repositorios separados.

- **AEMET + Normalización + SQL + Tests**

Consumo de API externa, persistencia relacional, validación Pydantic y 97 tests pasados.

El resultado es una base funcional que puede seguir evolucionando.

Proyecto 2 = Flask + JSON + AEMET + vistas HTML. — Proyecto 3 = FastAPI + SQLAlchemy + Pydantic + ETL + CRUD + trazabilidad.

Próximos pasos

- Separar `normalizer_service.py` en módulos más pequeños
- Mejorar el frontend principal con gráficas de evolución meteorológica
- Crear un sistema de histórico más completo
- Unificar criterios entre Flask y FastAPI
- Añadir autenticación más robusta
- Despliegue en la nube
- Mejorar la gestión de caché para evitar llamadas innecesarias a AEMET
- Limitar predicción horaria a próximas 12 o 24 horas